

Execute an Azure Cosmos DB Stored Procedure Using the .NET SDK

Summary:-

In this lab, you will execute an Azure Cosmos DB stored procedure from a .NET console application. Instead of inserting documents directly using the SDK, the application passes the order and audit information to the stored procedure, which performs the document creation inside the Azure Cosmos DB container.

Let's Start:-

After completing this lab, you will be able to:

- Connect a .NET application to Azure Cosmos DB.
- Execute a stored procedure using the Azure Cosmos DB .NET SDK.
- Pass input parameters to a stored procedure.
- Receive and verify the stored procedure response.

Lab Steps:-

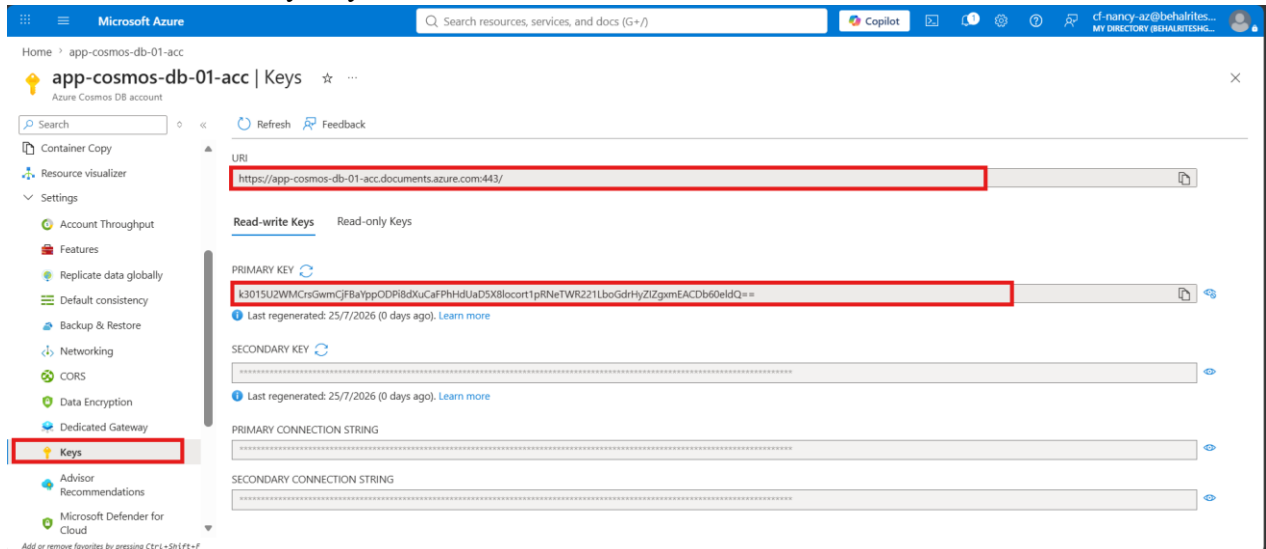
Step 1 – Prepare the Azure Cosmos DB .NET Project

- **Reference:** This lab uses the .NET console application, Azure Cosmos DB account, database, and container created in the previous lab.
- For that **create** Azure Cosmos DB account by selecting **Azure Cosmos DB for NoSQL** from the recommended APIs.
- Configure:
Workload type: Learning, **Region:** EAST US/ EAST US 2, **Capacity Mode:** Provisioned Throughput
- Enable **Apply Free Tier Discount**. Click **Next**.
- Ensure Geo-Redundancy and Multi-region Writes is **Disabled**. Click **Next**.
- Select **Enable access from all networks**. Click on **Review + Create** to create.
- From the left pane, select **Data Explorer**.
- Create a **Database** named **orders-db**.
- Inside the database, create a **Table** named **Orders** with
Partition Key: /customerId
- Open **Visual Studio Code**. Create a new project folder.
- Press **Ctrl + Shift + P** to open the **Command Palette**.
- Select **Show and Run Commands** > **.NET: New Project** > **Console App**(Rename it as cosmosdb-app) > **Default Directory** > **.slnx** > **Create Project**.
- **Open** the integrated terminal. Run:

```
cd .\storedProcedure\  
dotnet add package Microsoft.Azure.Cosmos
```

dotnet add package Newtonsoft.Json

- Paste the given **Program.cs** code.
- Configure the following values in **Program.cs**:
 - Endpoint
 - Primary Key



- Database ID: orders-db
- Container ID: Orders
- Partition Key: /customerId
- Open the following project files:
 - Program.cs**
 - storedProc-app.csproj**
 - storedProc-app.slnx**
- This step prepares the .NET application to connect to Azure Cosmos DB.

Step 2 – Add the Application Code

- Replace the existing **Program.cs** code with the provided **Program.cs** file.

```
1 using Microsoft.Azure.Cosmos;  
2 using Microsoft.Azure.Cosmos.Scripts;  
3 using Newtonsoft.Json;  
4  
5 string Endpoint="https://app-cosmos-db-01-acc.documents.azure.com:443/";  
6 string Key="k3015U2wMCrsGwmCjFbYppODPi8dXuCaFPhdUaD5X8l0cort1pRNeTW221LboGdrHyZIZgxmEACDb60e1dQ=="  
7 string DatabaseId = "orders-db";  
8 string ContainerId = "Orders";  
9 string PartitionKeyPath = "/customerId";  
10 string sprocId = "placeOrder";  
11  
12 var client = new CosmosClient(Endpoint, Key);  
13  
14 Container container = client.GetContainer(DatabaseId, ContainerId);  
15  
16 var customerId = "C004";  
17 var orderId = $"ord-001";  
18 var auditId = "audit-001";  
19  
20 var order = new  
21 {  
22     id = orderId,  
23     customerId = customerId,  
24     type = "order",  
25     total = 199.00  
26 };  
27  
28 var audit = new
```

- Save the file.
- This step adds the logic required to execute the stored procedure from the .NET application.

Step 3 – Create the Input Objects

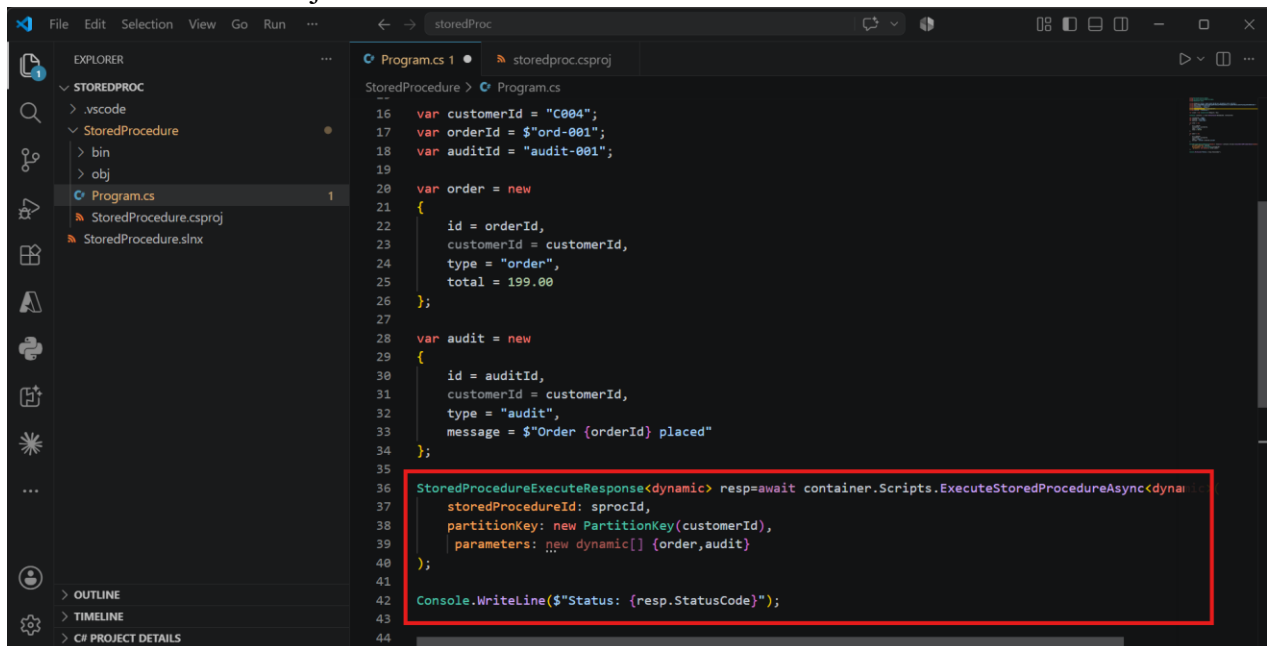
- Create an **Order** object containing the order details.
- Create an **Audit** object containing the audit information.

```
11  
12 var client = new CosmosClient(Endpoint, Key);  
13  
14 Container container = client.GetContainer(DatabaseId, ContainerId);  
15  
16 var customerId = "C004";  
17 var orderId = $"ord-001";  
18 var auditId = "audit-001";  
19  
20 var order = new  
21 {  
22     id = orderId,  
23     customerId = customerId,  
24     type = "order",  
25     total = 199.00  
26 };  
27  
28 var audit = new  
29 {  
30     id = auditId,  
31     customerId = customerId,  
32     type = "audit",  
33     message = $"Order {orderId} placed"  
34 };  
35  
36 StoredProcedureExecuteResponse<dynamic> resp=await container.Scripts.ExecuteStoredProcedureAsync<dynamic>(  
37     sprocId:  
38     partitionKey: new PartitionKey(customerId),  
39     parameters: new dynamic[] {order,audit}
```

- This step prepares the documents that will be passed as input parameters to the stored procedure.

Step 4 – Execute the Stored Procedure

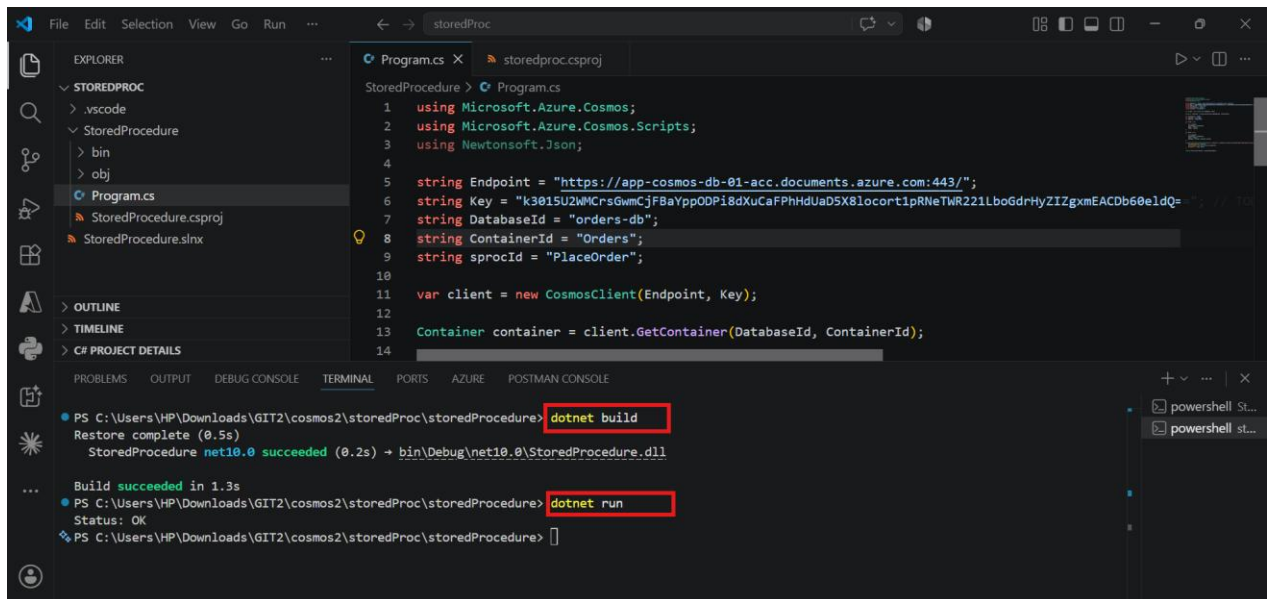
- Obtain a reference to the Azure Cosmos DB container.
- Use the **ExecuteStoredProcedureAsync()** method.
- Specify:
 - Stored Procedure ID (**PlaceOrder**)
 - Partition Key
 - Order object
 - Audit object



- Store the returned response.
- This step invokes the stored procedure and executes the server-side business logic inside Azure Cosmos DB.

Step 5 – Build and Run the Application

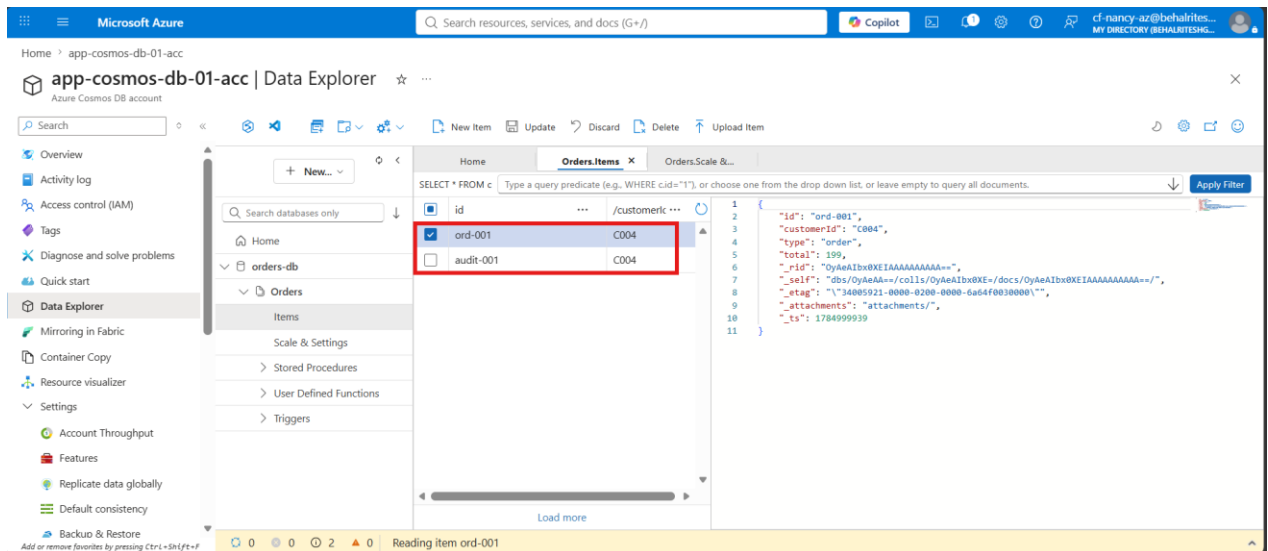
- Open the integrated terminal and run:
dotnet build
dotnet run
- Verify that the response status is **OK**.



- This step executes the stored procedure from the .NET application.

Step 6 – Verify the Documents

- Open the **Azure Portal**.
- Navigate to the **Azure Cosmos DB** account. Select **Data Explorer**.
- Open the **container** used in this lab. Refresh the **Items** node.
- Verify that:
 - One document contains the **Order** information.
 - One document contains the **Audit** information.



- This step confirms that the stored procedure created both documents successfully.

Verification

Verify the following:

- The .NET application connects successfully to Azure Cosmos DB.
- The **PlaceOrder** stored procedure executes successfully.
- The response status is **OK**.
- The Order document is created in the container.
- The Audit document is created in the container.
- Both documents belong to the specified partition key.